

Authorship Verification with Transformer Models : CS 7643

Peter Kirby Eric Wang Henry Luk
Georgia Institute of Technology

{pkirby6, ewang436, hlu6}@gatech.edu

Abstract

We present a method of fine-tuning a ModernBERT Bi-Encoder that achieves state of the art performance on the PAN21 authorship verification task. This improves on prior work with temperature-scaled InfoNCE loss, large batch size, and a tuned learning rate that is zero-shot transferred from a smaller model. Data augmentation in the form of truncating randomly rotated text is seen to be highly effective for minimizing generalization error. Longer context length is also shown to be key to performance in the PAN21 fanfiction setting. We compare this method to prior work, to a classical baseline, and to a cross-encoder classifier.

1. Introduction

1.1. Objective

Authorship verification [32] is a task that involves determining whether the same author wrote two different texts. Unlike authorship attribution, which involves identifying which text corpus from a list was written by the author of an unidentified text, authorship verification could also be used to predict that no such text pair has the same author, so identifying the most stylistically similar text is insufficient. As such, the task is challenging because of the need for generalizable representations of writing characteristics, not just categorizing author styles. We consider whether this task can be better solved as a binary classification problem or as a problem of learning vector representations of writing style where a similarity threshold is used to predict common authorship. To make meaningful comparisons and to understand what is contributing to performance, experiments are conducted with a smaller model to find appropriate configurations to optimize both approaches. Larger models are also fine-tuned to see the effect on performance and to attempt to build a competitive system in a specific authorship verification setting involving authors of fanfiction. The performance of fine-tuned transformer [59] models on the authorship verification task is compared to classical feature engineering methods and to prior work.

1.2. Related Work

Boenninghoff et al. [5] developed a bidirectional LSTM with attention [69] in 2019 by using Siamese networks [8] and pairwise contrastive loss [22] for authorship verification. This work was extended in 2021 with calibrated probability predictions and a system to withhold predictions based on out-of-domain detection [6]. This achieved the top performance on the PAN21 benchmark for fanfiction authorship verification [28]. An entry from Weerasinghe et al. [62] showed that an ensemble of classical feature engineering techniques can achieve competitive performance. Tyo et al. [57] used pairwise contrastive loss to fine-tune BERT [14] without strong results, which is useful for understanding what could improve performance. Peng et al. [47] designed a competitive entry for PAN21 that averaged encoded representations of a fine-tuned BERT from paired text snippets, compensating for the model’s limit on context length, while using two linear layers and softmax for binary classification. Gao et al. [17] developed a conceptually similar system to our approach, using BERT with InfoNCE loss [45], but this was in the context of semantic similarity [9]. Since BERT, transformer architectures have advanced significantly. For example, ModernBERT [61] was pretrained on more data and supports up to 8192 token context length while using rotary position embeddings [55], GeGLU activations [53], and pre-layer normalization [65]. Huertas-Tato et al. [23] fine-tuned RoBERTa-large [36], which has a 512 context length limit, for authorship similarity using supervised contrastive loss [29], a variation on InfoNCE loss, with modest results on PAN21 [38] without using fanfiction training data. Nguyen et al. [44] achieved very competitive PAN21 results using binary classification with BigBird [68], which supports longer context length. We explore classification and contrastive learning for authorship verification with transformer models smaller than BERT and with ModernBERT.

1.3. Motivation

Historical interest originally motivated authorship attribution research, such as the 1964 quantitative study by Mosteller and Wallace [40] examining authorship of the

Authorship Verification: Two Training Approaches

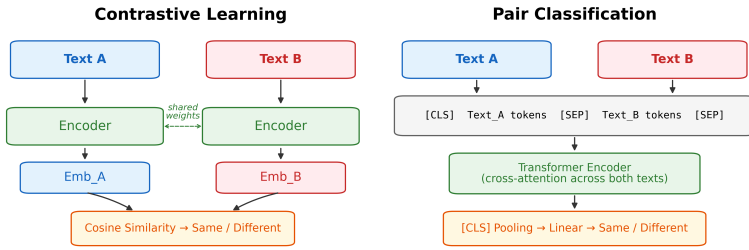


Figure 1. Comparison of our two training approaches. Contrastive learning (left) embeds each text independently and compares via cosine similarity. Pair classification (right) feeds both texts concatenated, enabling attention between texts.

Federalist Papers. For academic integrity, institutions could flag submitted work that may have false authorship claims as suspicious. In forensic linguistics, identifying the author behind threatening communications allows proper legal actions to be taken. On social media platforms, these systems could be used to flag new accounts created by banned users with the aim of improving the experience for ordinary users. Improvements in authorship verification techniques may be applicable to developing better automated systems for detecting AI-generated content, which can masquerade as real human opinion for commercial gain or political campaigns. We also hope to learn about fine-tuning transformer models on textual data in the process.

1.4. Dataset

Kestemont et al. [27] created the PAN20 cross-domain authorship verification dataset. The aim was to study authorship verification under the domain shift of different fandoms with a large training set to support neural networks. Each instance is a pair of English fanfiction document samples with integer author IDs and fandom names for each text. The training set has 148,000 same-author and 128,000 different-author pairs from 278,169 distinct authors overall, which is subsampled from a larger fanfiction.net dataset [4] to reduce overrepresentation of more common authors and fandoms. Each sample is limited to 21,000 characters; longer stories can produce multiple samples. Each pair has a same-author or different-author target label, which could also be derived from author IDs. The PAN20 test set had fandoms and authors from the training set, but PAN21 had unseen fandoms and authors for an open-set task. The test set has 10,000 different-author pairs and 9,999 same-author different-fandom pairs. This dataset continues to be influential in authorship verification research [7, 52, 58, 68]. The PAN20 dataset normalizes the spacing and punctuation; we re-corrected punctuation to use apostrophes for contractions, not quotation marks.

Model	Parameter Count	Layers	Dimensions
TinyBERT [25]	15M	4	312
DistilBERT-cased [51]	66M	6	768
BERT-base-cased [14]	110M	12	768
ModernBERT-base [61]	149M	22	768
ModernBERT-large [61]	395M	28	1024

Table 1. Transformer Model Size Summary.

2. Approach

2.1. Data Augmentation

Hugging Face hosts our preprocessed data. Our validation dataset is similar in construction to the official PAN21 test dataset, using 10,000 same-author different-fandom pairs and 10,000 different-author pairs such that authors and fandoms in the validation set are omitted from training data. Following Boenninghoff et al. [6], we convert the training data from fixed pairs into individual document rows with author IDs for dynamic pair recombination during training, yielding more varied same-author and different-author pairs, with billions of potential distinct negative pairs. We use PyTorch Metric Learning [42] to sample authors randomly, each time randomly selecting two document samples per author, which are distinct if an author has multiple samples. During training, the data collator rotates a document sample to start at a random word boundary before truncation by the tokenizer, creating different contiguous text views of the same document, analogous to random cropping, a standard data augmentation technique in image processing [10, 33]. The goal was to retain fine details for this task, but a recent pre-print [19] suggests a general advantage for text cropping over SimCSE-style dropout [17]. Random text rotation is a possibly novel data augmentation technique allowing equal length truncated spans at any position with little disturbance to the data. This can potentially create thousands of distinct views for one sample and millions of distinct pairs of strings for a pair of samples. Data augmentation agrees with this principle of *Deep Learning*: “The best way to make a machine learning model generalize better is to train it on more data.” [20, p. 240]

2.2. Transformer Models

We evaluate five transformer encoders of varying model capacity described in Table 1. Hugging Face was used to download weights for pre-trained transformers [63]. This coverage allowed us to test how well the pretraining of models at different sizes transferred to performance on the authorship verification task after fine-tuning. We use cased versions if available as this is known to improve authorship verification performance [34]. ModernBERT is also cased.

2.3. Pair Classification

As shown in Figure 1, the pair classification approach concatenates two truncated tokenized text samples as a sin-

gle input to the model, using a special token to mark the boundary, and predicts the same-author label from the joint sequence representation. For both classification and contrastive learning, the tokenizer [63] associated with the model is used, and the model’s embedding weights are included in fine-tuning. As a cross-encoder [50], it allows attention between tokens from both samples, so it can update intermediate representations at each layer using information from both texts rather than encoding each text independently. This often performs well, particularly with curated negatives [24], at the computational expense of the forward and backward pass scaling linearly for every pair. Each author in the batch has two identical positive pairs. One negative, different-author pair is created at random for each document sample in the batch, with two per author. After testing, our final configuration involves fine-tuning a pre-trained transformer model for sequence classification with weights from a pre-trained classification head [63] prior to the last linear layer, which produces a logit. We use a higher learning rate for the projection head [26] that is five times higher [11]. For prediction, the sigmoid output threshold that maximizes F1 on the validation set is used.

2.4. Contrastive Learning

Known as a bi-encoder architecture [8, 24], this approach encodes one embedding per document using the same model for all. Contrastive learning is recognized for its ability to learn useful representations [22] and for computational efficiency when scoring many pairs. Each document in a batch has one positive pair. Negative pairs include all documents by different authors in a batch. Training uses InfoNCE loss with temperature scaling [64] to emphasize harder examples. InfoNCE can be interpreted as a variant of cross-entropy loss that encourages low similarity for negative pairs and high similarity for positive pairs, maximizing a lower bound on mutual information [45] for same-class embeddings. It’s equivalent [50] to the NT-Xent loss [10] and Supervised Contrastive Loss [29] with one positive pair, and we use a SupCon implementation [42] for efficiency.

$$\mathcal{L}_i = -\log \frac{e^{s(\mathbf{z}_i, \mathbf{z}_{j+})/\tau}}{e^{s(\mathbf{z}_i, \mathbf{z}_{j+})/\tau} + \sum_{j^- \in \mathcal{N}_i} e^{s(\mathbf{z}_i, \mathbf{z}_{j-})/\tau}} \quad (1)$$

After testing, our final configuration applies mean pooling over the final hidden states of the encoder and doesn’t use a projection head. Similarity between document samples is computed using cosine similarity between the embeddings, allowing arbitrary pairs to be scored after encoding independently. For prediction, we select the cosine similarity threshold that maximizes F1 on the validation set.

2.5. TF-IDF Baseline

We used TF-IDF [54] (Text Frequency-Inverse Document Frequency) features combined with a logistic regres-

sion classifier to act as a classical baseline to compare against for authorship verification. Each document was represented as a sparse vector of word unigram and bigram features to capture word usage patterns. Training pairs were constructed in the same manner as our other pipelines. Using pairs of TF-IDF vectors, we used a feature representation based on absolute difference to capture stylistic differences between texts. The features were then used to train a logistic regression classifier to predict if two documents share an author. This approach serves as a useful baseline by leveraging lexical and stylistic signals but also is likely confounded with topic signals [4]. It is compared against transformer-based methods, which have the capability of capturing rich semantic and syntactic relationships. We use sklearn [46] for TF-IDF vectorization and logistic regression. We use PyTorch Lightning to organize our code, abstract some low-level details, and as a deep learning framework for classification and contrastive learning.

2.6. Hyperparameter Transfer

Hyperparameter sweeps with TinyBERT are described below as experiments. We use a form of μ Transfer [67] where hyperparameters are tuned on a small model for zero-shot transfer to larger models. For transfer across depth, Complete(d)P [39] with $\alpha = 1$ justifies making no adjustment for depth. In practice, we transfer only learning rate based on the change in width. This involves an adjustment factor of 0.4 going from width 312 (TinyBERT) to width 768 (for DistilBERT, BERT, and ModernBERT-base) and an adjustment factor of 0.3 going from width 312 (TinyBERT) to width 1024 (ModernBERT-large). For simplicity not all hyperparameters are tuned [18], so we don’t transfer weight decay, β_1 , β_2 , and ϵ for AdamW but instead use recommendations or defaults for each model. We do not apply weight decay to biases or layer normalization [14]. We use linear decay [2] with 10% warmup [21]. We use PagedAdamW8bit [13] for AdamW [37] to save VRAM. We also train models in BF16 precision and use the Flash Attention [12] implementation of attention to save VRAM.

2.7. Distributed Data Parallel

Sentence Transformers [50] implements GradCache [16] as CachedMultipleNegativesRankingLoss with support for a DDP [35] (distributed data parallel) strategy, something we implement using PyTorch Lightning’s DDP support. Embeddings are pooled across GPUs, with different microbatches processed by different GPUs. GradCache saves on VRAM to support larger batch sizes, which benefit contrastive learning with InfoNCE loss [10, 29]. DDP increases parallelism and reduces the wall clock time of training. We use 8x Nvidia RTX 4090 with 24GB VRAM each.

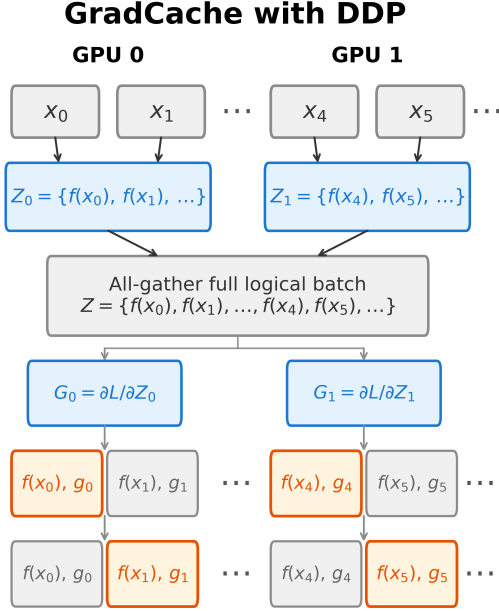


Figure 2. Distributed training with GradCache memory savings. An example where each x corresponds to a microbatch. Computation attached to the computational graph is shown in orange. Detached computation is shown in blue.

2.8. GradCache

GradCache [16] is implemented with a generalization to support any parameter-free PyTorch Metric Learning [42] loss function. We do a forward pass without gradient tracking, saving random state for idempotency, to compute embeddings Z for the logical batch. We compute the full-batch loss on the embeddings and save gradient $\partial L / \partial Z$ as gradient cache G . Activations, which otherwise scale with the logical batch size, are saved to memory only for sequentially processed microbatches of input features x , cached gradients g , and embeddings z recomputed from x with another forward pass using saved random state. Given that $\partial \ell / \partial z = g$, the dot product $\ell = z^\top g$ is calculated as the microbatch loss for backpropagation through the computational graph for the neural network function $z = f(x)$.

2.9. Challenges

We anticipated problems with the computation required for many experiments, and solutions to that problem as described in this paper were developed iteratively. We planned on renting vast.ai instances to be able to run experiments concurrently. We set up our GitHub repository [31] so that it could be cloned and an environment could be set up with a shell script that uses Conda. We used Hugging Face to download a preprocessed dataset. We used wandb.ai [3] to persist and organize data from experiments.

We planned on using the repository by Boenninghoff et al. [6] to preprocess the data, but we found that Boen-

ninghoff used a number of text preprocessing normalization techniques that were specifically designed for a setting where custom pre-trained embeddings for this dataset were used for a bidirectional LSTM [5]. Our transformer models had tokenizers that were designed for non-normalized text, and it was possible that details of the data required for authorship verification would be lost. We kept Boenninghoff et al.’s approach of using Parquet for convenience and of using dynamic recombination.

We began by thinking that we would fork SimCSE [17] and adapt the code to the authorship verification task, different models, different loss functions, and so on. Trying to organize and work through all the code became unwieldy, especially due to the need to handle a high number of experiment configurations and even entire different training approaches. PyTorch Lightning solved these problems. New experiment arguments merely had to be added to a module’s init function and could immediately be used from the command line or a YAML config file. Most importantly, it encouraged highly organized code that made it easy to work together on different model modules while sharing other logic such as data sampling and evaluation.

Another useful library that we found as we progressed along was PyTorch Metric Learning. This allowed us to test many different loss functions. We initially surmised that evaluating many innovative losses from the literature would be important for performance because many publications focus on novel contributions to loss function research. The experiments led us to InfoNCE [64], which has a simple interpretation based on mutual information [45].

3. Experiments and Results

3.1. Evaluation Methodology

We chose F1 as the primary metric because it’s readily available from prior work and because optimizing for it encourages precision and recall, while accuracy can conceal unbalanced performance on positives and negatives. The threshold for predictions and best checkpoint across epochs were selected by validation F1. Hyperparameters are tuned on validation F1 to avoid test set leakage. Tuned models are compared to each other and prior work with test F1.

3.2. Classification Experiments

3.2.1 Encoder Freezing

Table 2 shows a large drop in performance with a frozen encoder. Only 1,536 parameters via the linear classification head are trainable. The performance drop-off illustrates that BERT’s pre-trained representations are a weaker signal for authorial style despite their strengths in general language understanding. This follows our intuition that significant training through the fine-tuning process is required to adjust representations to represent authorial style well.

	Trainable Parameters	Test Acc	Test F1
Fully Fine-tuned	109M	0.761	0.708
Frozen Encoder	1.5K	0.613	0.441

Table 2. BERT classification, 3 epochs, not thresholded for F1.

Pooling	Test Acc	Test F1
CLS	0.761	0.708
Mean	0.761	0.708
CLS-Concat (last 4 layers)	0.757	0.704

Table 3. BERT classification, 3 epochs, not thresholded for F1.

Batch Size	lr=2e-5	lr=4e-5	lr=8e-5
8	0.740	0.743	0.736
16	0.737	0.739	0.738
32	0.736	0.738	0.737
64	0.734	0.734	0.738
128	0.731	0.733	0.736
256	0.724	0.734	0.735

Table 4. Validation F1 for TinyBERT classification, 10 epochs.

3.2.2 Pooling Strategy

Pooling variations between CLS and mean had marginal impact on classification. This follows our intuition that the next sentence prediction task of BERT encourages a strong feature representation of the CLS embedding through the transformer layers themselves. Since CLS token performance with BERT works well, further experiments use a model for sequence classification with weights from a pre-trained classification head on the CLS token [63].

3.2.3 Batch Size and Learning Rate

The smaller TinyBERT is fine-tuned for 10 epochs. A prediction threshold is selected on validation F1. A grid search across learning rates and batch sizes is used to find hyperparameters for μ Transfer [67] to larger models. Classification models see a benefit from smaller batch size for a fixed epoch budget with more small optimizer steps.

3.3. Contrastive Experiments

3.3.1 Loss Function

Our literature review can be interpreted as revealing two different ecosystems of contrastive learning research. One surveyed by Kevin Musgrave uses a few small datasets with a long lineage to explore novel techniques but suffers from replication issues [41]. Another uses practical large datasets such as ImageNet [10] and Wikipedia [17] where InfoNCE loss is extended to emphasize harder instances [64], include multiple positives [10, 29], and function practically at larger batch sizes [16]. Our results align with the latter. There’s a strong theoretical literature behind InfoNCE [45, 48] that is relatively lacking for the alternatives. Table 5 shows that a pairwise contrastive loss [66], which has been used in research on authorship verification [5, 57], performed worse than losses that can consider all negatives in a batch such

Loss Function	Val Acc	Val F1
InfoNCE	0.753	0.774
Circle	0.732	0.764
Multi-Similarity	0.714	0.748
Semi-Hard Contrastive	0.704	0.734
Softtriple	0.500	0.667
Proxyanchor	0.500	0.667

Table 5. TinyBERT, batch size 256, learning rate 2e-5, 10 epochs.

Batch Size	lr=2e-4	lr=4e-4	lr=8e-4
256	0.804	0.799	0.790
512	0.804	0.807	0.796
1024	0.797	0.805	0.787

Table 6. Validation F1 for TinyBERT contrastive, 10 epochs.

Batch Size	lr=2e-4	lr=4e-4	lr=8e-4
512	0.855	0.852	0.845
1024	0.854	0.856	0.851
2048	0.850	0.853	0.855

Table 7. Validation F1 for TinyBERT contrastive, 40 epochs.

Temperature	0.01	0.03	0.05	0.07	0.1
Val F1	0.8557	0.8556	0.851	0.835	0.804

Table 8. Validation F1 for TinyBERT contrastive, 40 epochs, 1024 batch size, 4e-4 learning rate, with a two-layer projection head and mean first-last layer pooling.

Configuration	Val Acc	Val F1
Projection Head + Mean First-Last Layer Pooling	0.851	0.856
Mean First-Last Layer Pooling only	0.853	0.857
Projection Head + Mean Pooling	0.857	0.860
Mean Pooling only	0.861	0.863
Mean Pooling only, no random text spans (6 epochs)	0.713	0.740
Mean Pooling only, no random text spans, 40 epochs	0.682	0.727

Table 9. Ablation results for TinyBERT contrastive with batch size 1024 learning rate 4e-4, temperature 0.01 over 40 epochs.

as InfoNCE, Circle [56], and Multi-Similarity [60]. Soft-Triple [49] and ProxyAnchor [30], which attempt to learn embeddings per class, fail in this setting that has 238,815 author classes in the training set used.

3.3.2 Batch Size and Learning Rate

TinyBERT is fine-tuned for 10 epochs and for 40 epochs, again using grid search across learning rates and batch sizes to find hyperparameters for μ Transfer to larger models. Contrastive learning benefits from larger batch sizes [10, 29] that provide a better estimate for the negative pair term by sampling more of the negative pair population [45, 48]. This benefit reaches a limit for a fixed epoch budget due to fewer optimizer steps at larger batch sizes.

3.3.3 Temperature

Changing temperature from 0.05 to 0.01 is found to improve performance by putting more emphasis on harder examples. Since the larger models have more capacity than TinyBERT to learn from hard examples, 0.01 temperature is selected.

Model	Objective	Test Accuracy	Test F1
TF-IDF Baseline	Classification	0.611	0.698
TinyBERT	Classification	0.745	0.761
DistilBERT-cased	Classification	0.684	0.725
BERT-base-cased	Classification	0.787	0.797
ModernBERT-base	Classification	0.653	0.706
ModernBERT-large	Classification	0.703	0.742
TinyBERT	Contrastive	0.812	0.812
DistilBERT-cased	Contrastive	0.877	0.874
BERT-base-cased	Contrastive	0.891	0.881
ModernBERT-base	Contrastive	0.900	0.898
ModernBERT-large	Contrastive	0.913	0.911

Table 10. Context length 512, epoch 10 results for tuned models.

Context Length	Batch Size	Test Accuracy	Test F1
512	1024	0.926	0.925
1024	1024	0.952	0.955
2048	2048	0.976	0.975
4096	4096	0.984	0.984

Table 11. Contrastive ModernBERT-large trained 40 epochs at 1.2×10^{-4} learning rate.

Model	Test F1
STAR [38]	0.79
tyo21 [57]	0.791
weerasinghe21 [62]	0.916
peng21 [47]	0.917
embarcaderoruiz21 [15]	0.934
BigBird Cross-Encoder [68]	0.944
boenninghoff21 [6]	0.952
ModernBERT Bi-Encoder (Ours)	0.984

Table 12. Comparison to prior work.

3.3.4 Ablation Testing

Table 9 shows how ablation testing revealed better results with mean pooling only. Appendix A shows configurations involving a projection head and pooling with the average of the first and last hidden layers [17] that initially showed positive results with a lower learning rate and fewer epochs. Because this reverses at tuned learning rates and more epochs, we use the configuration with mean pooling on the last layer only and no projection head for contrastive learning. Intuitively this is interpreted as forcing the model to use all the transformer layers and to use transformers directly for the representation with respect to every token. The performance drop and overfitting without random text spans show the effectiveness of the data augmentation.

3.4. Model Comparison

As expected, larger models outperform smaller models with contrastive learning. This isn’t true for classification where the BERT model outperformed the larger ModernBERT models. ModernBERT chose to drop the next sentence prediction task from pre-training that was the target of the representation from the CLS token. We suggest that pre-training on the next sentence prediction task was important for pre-training weights that allowed pre-trained representations to fine-tune well to adapt to our task.

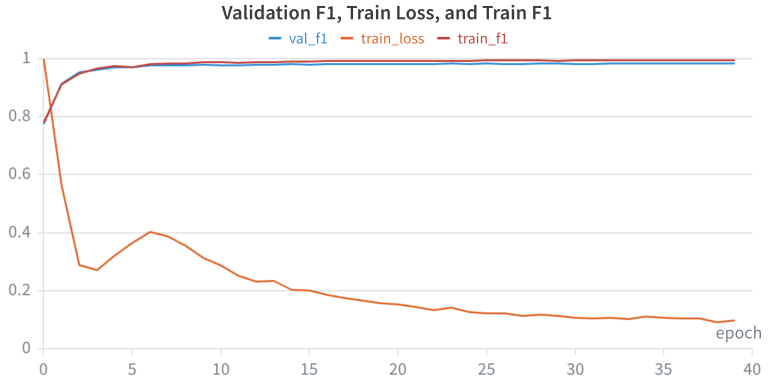


Figure 3. ModernBERT-large (4096 token context length).

3.5. Context Length and Batch Size

Context length 1024 instead of 512 increases performance. It’s left to future work to investigate whether training over more tokens using longer context length effectively means more data that supports choosing a larger batch size.

3.6. Comparison to Prior Work

Considered as a baseline here, STAR uses RoBERTa-large and supervised contrastive loss to create a zero-shot model that is not fine-tuned on fanfiction data. Tyo et al. use BERT and pairwise contrastive loss while fine-tuning on fanfiction data, but results of 0.791 are lower than our TinyBERT at 0.863. Peng et al.’s concatenated input BERT model with classification achieves results at 0.917 that exceed our own with BERT at 0.881, which can be attributed to using more than only 512 tokens across two texts. The results with BigBird Cross-Encoder and Boenninghoff et al.’s bidirectional LSTM, neither of which are constrained by 512 length context, also show the value of longer context on the fanfiction data that has 21,000 characters per document. Our ModernBERT Bi-Encoder with 4096 token context length achieves state-of-the-art performance. We consider our research successful based on what we learned about fine-tuning transformer models.

4. Discussion

As shown in Figure 3, our largest model’s loss exhibits the double descent pattern [43] instead of a classical bias-variance trade-off [1]. Low generalization error instead of overfitting may be attributed primarily to data augmentation. Longer context length, using μ Transfer for training a larger model, and InfoNCE loss with tuned batch size and learning rate contribute the most to performance.

5. Acknowledgements and Work Division

See Appendix B and Table 16.

References

- [1] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019. 6
- [2] Shane Bergsma, Nolan Simran Dey, Gurpreet Gosal, Gavia Gray, Daria Soboleva, and Joel Hestness. Straight to zero: Why linearly decaying the learning rate to zero works best for llms. In *International Conference on Learning Representations*, 2025. 3
- [3] Lukas Biewald. Experiment tracking with weights and biases, 2020. Software available from wandb.com. 4
- [4] Sebastian Bischoff, Niklas Deckers, Marcel Schliebs, Ben Thies, Matthias Hagen, Efstathios Stamatatos, Benno Stein, and Martin Potthast. The Importance of Suppressing Domain Style in Authorship Analysis. *CoRR*, abs/2005.14714, May 2020. 2, 3
- [5] Benedikt Boenninghoff, Steffen Hessler, Dorothea Kolossa, and Robert M. Nickel. Explainable authorship verification in social media via attention-based similarity learning. In *2019 IEEE International Conference on Big Data (Big Data)*, pages 36–45, 2019. 1, 4, 5
- [6] Benedikt Boenninghoff, Robert M. Nickel, and Dorothea Kolossa. O2D2: Out-of-distribution detector to capture undecidable trials in authorship verification. In *CLEF 2021 Labs and Workshops, Notebook Papers*, volume 2936 of *CEUR Workshop Proceedings*, Bucharest, Romania, 2021. CEUR-WS.org. Notebook for PAN at CLEF 2021. 1, 2, 4, 6
- [7] Florin Brad, Andrei Manolache, Elena Burceanu, Antonio Barbalau, Radu Tudor Ionescu, and Marius Popescu. Rethinking the authorship verification experimental setups. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5634–5643, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. 2
- [8] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, and Roopak Shah. Signature verification using a “Siamese” time delay neural network. In *Advances in Neural Information Processing Systems*, volume 6, pages 737–744. Morgan Kaufmann, 1993. 1, 3
- [9] Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. SemEval-2017 task 1: Semantic textual similarity – multilingual and cross-lingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada, 2017. Association for Computational Linguistics. 1
- [10] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1597–1607. PMLR, 2020. 2, 3, 5
- [11] Anton Chernyavskiy, Dmitry Ilvovsky, and Preslav Nakov. Transformers: “the end of history” for natural language processing? In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 677–693. Springer, 2021. 3
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, 2022. 3
- [13] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization. In *International Conference on Learning Representations*, 2022. 3
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. 1, 2, 3
- [15] Daniel Embarcadero-Ruiz, Helena Gómez-Adorno, Alberto Embarcadero-Ruiz, and Gerardo Sierra. Graph-based siamese network for authorship verification. *Mathematics*, 10(2):277, 2022. 6
- [16] Luyu Gao, Yunyi Zhang, Jiawei Han, and Jamie Callan. Scaling deep contrastive learning batch size under memory limited setup. In *Proceedings of the 6th Workshop on Representation Learning for NLP*, pages 316–321, Online, Aug. 2021. Association for Computational Linguistics. 3, 4, 5
- [17] Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6894–6910, Online and Punta Cana, Dominican Republic, Nov. 2021. Association for Computational Linguistics. 1, 2, 4, 5, 6
- [18] Varun Godbole, George E. Dahl, Justin Gilmer, Christopher J. Shallue, and Zachary Nado. Deep learning tuning playbook, 2023. Version 1.0. 3
- [19] Rita González-Márquez, Philipp Berens, and Dmitry Kobak. Cropping outperforms dropout as an augmentation strategy for self-supervised training of text embeddings, 2026. 2
- [20] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. 2
- [21] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large mini-batch sgd: Training imagenet in 1 hour. In *arXiv preprint arXiv:1706.02677*, 2017. 3
- [22] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’06)*, volume 2, pages 1735–1742. IEEE, 2006. 1, 3
- [23] Javier Huertas-Tato, Alejandro Martin, and David Camacho. Understanding writing style in social media with a supervised contrastively pre-trained transformer, 2023. 1
- [24] Samuel Humeau, Kurt Shuster, Marie-Anne Lachaux, and Jason Weston. Poly-encoders: Architectures and pre-training strategies for fast and accurate multi-sentence scoring. In *International Conference on Learning Representations*, 2020. 3
- [25] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for natural language understanding. In Trevor

- Cohn, Yulan He, and Yang Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, Online, Nov. 2020. Association for Computational Linguistics. 2
- [26] Mandar Joshi, Omer Levy, Luke Zettlemoyer, and Daniel Weld. BERT for coreference resolution: Baselines and analysis. In *Proceedings of EMNLP-IJCNLP*, pages 5803–5808. Association for Computational Linguistics, 2019. 3
- [27] Mike Kestemont, Enrique Manjavacas, Ilia Markov, Janek Bevendorff, Matti Wiegmann, Efstathios Stamatatos, Martin Potthast, and Benno Stein. Overview of the cross-domain authorship verification task at PAN 2020. In *Working Notes of CLEF 2020 – Conference and Labs of the Evaluation Forum*, volume 2696 of *CEUR Workshop Proceedings*, Sept. 2020. 2
- [28] Mike Kestemont, Enrique Manjavacas, Ilia Markov, Janek Bevendorff, Matti Wiegmann, Efstathios Stamatatos, Benno Stein, and Martin Potthast. Overview of the cross-domain authorship verification task at PAN 2021. In *Proceedings of the Working Notes of CLEF 2021 - Conference and Labs of the Evaluation Forum*, volume 2936 of *CEUR Workshop Proceedings*, pages 1743–1759. CEUR-WS.org, 2021. 1
- [29] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 18661–18673. Curran Associates, Inc., 2020. 1, 3, 5
- [30] Sungyeon Kim, Dongwon Kim, Minsu Cho, and Suha Kwak. Proxy anchor loss for deep metric learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3238–3247, 2020. 5
- [31] Peter Kirby, Eric Wang, and Henry Luk. Contrastive AV: Authorship verification through contrastive learning with modern embedding models. <https://github.com/petekirby/contrastive-av>, 2026. GitHub repository. 4
- [32] Moshe Koppel and Jonathan Schler. Authorship verification as a one-class classification problem. In *Proceedings of the Twenty-First International Conference on Machine Learning*. ACM, 2004. 1
- [33] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012. 2
- [34] Miles Q. Li, Benjamin C. M. Fung, Shih-Chia Huang, and Claude Fachkha. How to tame pre-trained transformers for authorship verification, 2025. SSRN preprint, posted April 24, 2025. 2
- [35] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. Pytorch distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13(12):3005–3018, 2020. 3
- [36] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019. 1
- [37] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. 3
- [38] Pablo Miralles-González, Javier Huertas-Tato, Alejandro Martín, and David Camacho. LLM one-shot style transfer for authorship attribution and verification, 2025. 1, 6
- [39] Bruno Mlodozieniec, Pierre Ablin, Louis Béthune, Dan Busbridge, Michal Klein, Jason Ramapuram, and Marco Cuturi. Completed hyperparameter transfer across modules, width, depth, batch and duration. In *International Conference on Learning Representations*, 2026. 3
- [40] Frederick Mosteller and David L. Wallace. *Inference and Disputed Authorship: The Federalist*. Addison-Wesley, Reading, MA, 1964. 1
- [41] Kevin Musgrave, Serge Belongie, and Ser-Nam Lim. A metric learning reality check. In *Computer Vision – ECCV 2020*, pages 681–699. Springer, 2020. 5
- [42] Kevin Musgrave, Serge J. Belongie, and Ser-Nam Lim. Pytorch metric learning. *arXiv preprint arXiv:2008.09164*, 2020. 2, 3, 4
- [43] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003, 2021. 6
- [44] Trang Nguyen, Charlie Dagli, Kenneth Alperin, Courtland Vandam, and Elliot Singer. Improving long-text authorship verification via model selection and data tuning. In Stefania Degaetano-Ortlieb, Anna Kazantseva, Nils Reiter, and Stan Szpakowicz, editors, *Proceedings of the 7th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 28–37, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics. 1
- [45] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 1, 3, 4, 5
- [46] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. 3
- [47] Zeyang Peng, Leilei Kong, Zhijie Zhang, Zhongyuan Han, and Xu Sun. Encoding text information by pre-trained model for authorship verification. In *CLEF 2021 – Conference and Labs of the Evaluation Forum*, CEUR Workshop Proceedings. CEUR-WS.org, 2021. 1, 6
- [48] Ben Poole, Sherjil Ozair, Aaron Van Den Oord, Alex Alemi, and George Tucker. On variational bounds of mutual information. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5171–5180. PMLR, 2019. 5

- [49] Qi Qian, Lei Shang, Baigui Sun, Juhua Hu, Hao Li, and Rong Jin. Softtriple loss: Deep metric learning without triplet sampling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6450–6458, 2019. 5
- [50] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. 3
- [51] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, 2020. 2
- [52] Jitkapat Sawatphol, Can Udomcharoenchaikit, and Sarana Nutanong. Addressing topic leakage in cross-topic evaluation for authorship verification. *Transactions of the Association for Computational Linguistics*, 12:1363–1377, 2024. 2
- [53] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. 1
- [54] Karen Spärck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972. 3
- [55] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021. 1
- [56] Yifan Sun, Changmao Cheng, Yuhao Zhang, Chi Zhang, Liang Zheng, Zhongdao Wang, and Yichen Wei. Circle loss: A unified perspective of pair similarity optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020. 5
- [57] Jacob Tyo, Bhuwan Dhingra, and Zachary Lipton. Siamese bert for authorship verification. In *Proceedings of the Working Notes of CLEF 2021 – Conference and Labs of the Evaluation Forum*, volume 2936 of *CEUR Workshop Proceedings*, pages 2169–2177, Bucharest, Romania, 2021. CEUR-WS.org. 1, 5, 6
- [58] Jacob Tyo, Bhuwan Dhingra, and Zachary C. Lipton. Valla: Standardizing and benchmarking authorship attribution and verification through empirical evaluation and comparative analysis. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 649–660, Nusa Dua, Bali, 2023. Association for Computational Linguistics. 2
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. 1
- [60] Xun Wang, Xintong Han, Weilin Huang, Dengke Dong, and Matthew R. Scott. Multi-similarity loss with general pair weighting for deep metric learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5022–5030, 2019. 5
- [61] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Nathan Cooper, Griffin Adams, Jeremy Howard, and Iacopo Poli. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference, 2024. 1, 2
- [62] Janith Weerasinghe, Rhia Singh, and Rachel Greenstadt. Feature vector difference based authorship verification for open-world settings. In *CLEF 2021 – Conference and Labs of the Evaluation Forum*, CEUR Workshop Proceedings. CEUR-WS.org, 2021. 1, 6
- [63] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020. 2, 3, 5
- [64] Zhirong Wu, Yuanjun Xiong, Stella X. Yu, and Dahua Lin. Unsupervised feature learning via non-parametric instance discrimination. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018. 3, 4, 5
- [65] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533. PMLR, 2020. 1
- [66] Harrison Xuan, Abby Stylianou, and Robert Pless. Improved embeddings with easy positive triplet mining. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 2474–2482, 2020. 5
- [67] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems*, 2021. 3, 5
- [68] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, volume 33, pages 17283–17297, 2020. 1, 2, 6
- [69] Peng Zhou, Wei Shi, Jun Tian, Zhenyu Qi, Bingchen Li, Hongwei Hao, and Bo Xu. Attention-based bidirectional long short-term memory networks for relation classification. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 207–212, Berlin, Germany, 2016. Association for Computational Linguistics. 1

Negatives Per Positive	Binary Cross Entropy	Focal Loss
1	0.743	0.740
2	0.739	0.742
4	0.741	0.744

Table 13. Validation F1 for TinyBERT classification with batch size 8, learning rate 4e-5, 10 epochs.

Pooling	Val Acc	Val F1
Mean First+Last Hidden Layer	0.749	0.7749
Mean Last Hidden Layer	0.757	0.7745
CLS	0.751	0.772
CLS Concat (last 4 layers)	0.731	0.765
Max Last Hidden Layer	0.731	0.759

Table 14. Validation Accuracy and F1 for TinyBERT contrastive, batch size 256, learning rate 2e-5, 10 epochs.

Projection Head	Val Acc	Val F1
Residual + Linear, LayerNorm, GELU, Linear	0.767	0.779
Linear, LayerNorm, GELU, Linear	0.758	0.777
None (SentenceBERT)	0.758	0.774
Linear, Linear	0.748	0.774
Linear, Tanh (SimCSE)	0.748	0.772

Table 15. Validation Accuracy and F1 for TinyBERT contrastive, batch size 256, learning rate 2e-5, 10 epochs.

A. Additional Experiments

These configurations were not pursued further.

A.1. Negatives Per Positive and Focal Loss

The classification model uses an equal number of positive and negative examples, one of each for every instance of data processed. This experiment (Table 13) explored using multiple negative pairs per anchor instead, sampled randomly, along with the focal loss using gamma 1 for harder negatives and class-balanced alpha. Given 150% higher computational cost to process 3 extra pairs for a negligible benefit, neither focal loss nor multiple negatives per positive were adopted for the classification model.

A.2. Pooling Strategy

Averaging over both the first hidden layer and the final hidden layer was used in SimCSE. Taking the average of just the last hidden layer performed better in ablation testing after tuning learning rate and batch size.

A.3. Projection Head

A two-layer projection head using LayerNorm and GELU, wrapped in a residual, seemed to perform slightly better when learning rate was undertuned. Using no projection head performed better in ablation testing after tuning learning rate and batch size.

B. Project Code Acknowledgements

The complete source code for this project is publicly available on GitHub at <https://github.com/petekirby/contrastive-av>.

Our preprocessed dataset is at https://huggingface.co/datasets/peterkirby/pan2020_dict_author_fandom_doc.

The repository was originally forked from the SimCSE repository, <https://github.com/princeton-nlp/SimCSE>. Although the present project retains almost none of the original codebase, after switching over to using the PyTorch Lightning library, the `mean_first_last` pooling method and the SimCSE-style projection head in `TransformerEmbeddingModel` still are adapted from SimCSE.

Preprocessing code and evaluation utilities in the `preprocessing/` and `helper_functions/` folders are adapted from the PAN 2020/2021 authorship verification codebase by Boenninghoff et al., https://github.com/boenninghoff/pan_2020_2021_authorship_verification.

This project makes extensive use of PyTorch Lightning, <https://github.com/Lightning-AI/pytorch-lightning>, and PyTorch Metric Learning, <https://github.com/KevinMusgrave/pytorch-metric-learning>. These are used as libraries, not by adapting code. PyTorch Lightning is a deep learning framework that structures our code. The PyTorch Metric Learning library provides loss functions, `mperclass` sampling, and a utility for processing sampled data into triplets in our collator.

The `RandContext` function implementation in `torch_utils` is adapted from GradCache, https://github.com/luyug/GradCache/blob/main/src/grad_cache/context_managers.py.

The mean pooling implementation is adapted from the Hugging Face Transformers usage example, <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2#usage-huggingface-transformers>.

The max pooling implementation is adapted from the Sentence Transformers pooling module, https://github.com/huggingface/sentence-transformers/blob/main/sentence_transformers/models/Pooling.py.

The last-token pooling implementation is adapted from the Qwen3 Embedding Transformers usage example, <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B#transformers-usage>.

Student	Contributed Aspects	Details
Peter Kirby	Data pipeline, contrastive learning infrastructure, report writing	Pre-processed PAN20/21 data and hosted it on Hugging Face. Implemented TransformerEmbedding-Model with multiple pooling and projection head options. Built the contrastive learning pipeline with PyTorch Lightning alongside various integrated loss functions. Implemented GradCache-style micro-batch training and an evaluation framework with threshold calibration. Ran contrastive experiments.
Eric Wang	Classification infrastructure, classification pair collator, report writing	Implemented classification collators with triplet mining and random span augmentation. Built TransformerClassificationModule (a Lightning module with CE loss, encoder freezing, and param-group optimization). Ran experiments across BERT-family models. Conducted ablation experiments on pooling strategy, encoder freezing, and contrastive versus classification comparison.
Henry Luk	Baseline module, TF-IDF with logistic regression, report writing	Implemented TF-IDF feature extraction and logistic regression baseline. Built BaselineModelModule. Evaluated classical model performance.

Table 16. Summary of individual contributions to the project.